Technologies Overview for ReMatch (MERN Based, NLP- Powered Resume Parsing & Ranking)

React.js

- Component-Based Architecture: Reacts virtual DOM and composable components will let us break our UI into small reusable pieces
- JSX Syntax : Writing markup directly in JavaScript makes component logic and presentation co-locate, also speeding up development and improving readability and maintainability.

Node.js

- Event-Driven, Non-Blocking I/O: Ideal for handling multiple concurrent parsing.
- JavaScript across stack: enables sharing of utility modules between backend and frontend reducing context
- NPM Eco system

Express.js

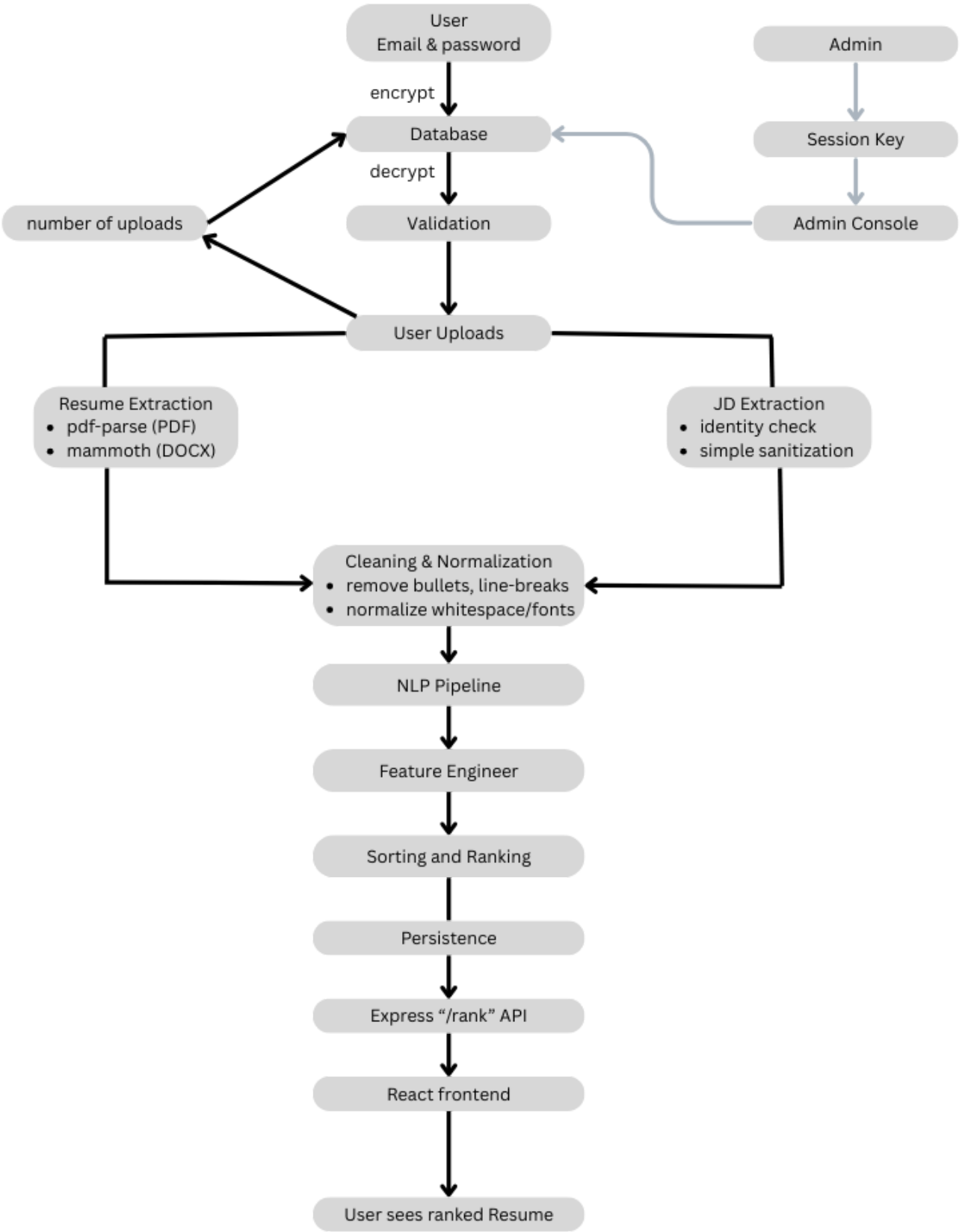
- Minimal and unopinionated framework: We will define only the routes and middlewares we need.
- Middleware: multer, JSON body parsing, authentication checks and custom error handlers in clear sequence.
- Easy to integrate ‘cors’ settings for our React frontend.

Mongoose

- Schema Definitions
- Validation & Hooks: run pre-save hooks for text-cleanup

NLP & Text Processing (JavaScript Native)

- Text Extraction: Use ‘pdf-parse’ for pdfs and ‘mammoth’ for docx.
- Skill Matching & Relevance Scoring: A ‘natural’ to compare the stemmed versions
- TF-IDF Matching: with natural.TfIdf
- NER & Skill Extraction: using ‘compromise’ build own custom matching rules.
- Scoring Logic: everything combined into score function.


User
Email & password
Admin
encrypt
Database
Session Key
decrypt
number of uploads
Validation
Admin Console
User Uploads
Resume Extraction
pdf-parse (PDF)
mammoth (DOCX)
JD Extraction
identity check
simple sanitization
Cleaning & Normalization
remove bullets, line-breaks
normalize whitespace/fonts
NLP Pipeline
Feature Engineer
Sorting and Ranking
Persistence
Express “/rank” API
React frontend
User sees ranked Resume